Progress Report: Sprint 3

CS.028 Optimizing Research Software Codes - 9 November 2025
Kathryn Butler, Michael McAllister, Joseph Schaab

Executive Summary

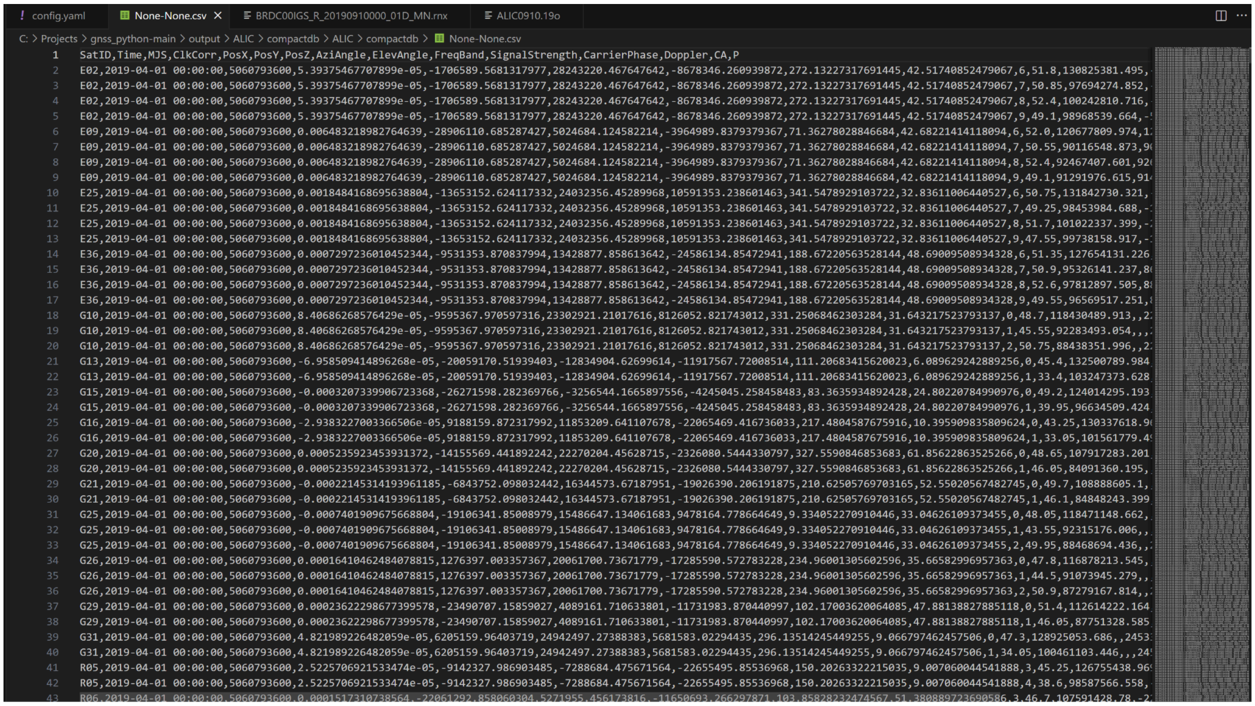
- We've expanded our testing data so we can now calculate the average runtime of each function, the total runtime of each function, and the number of times each function was called.
- We have also created documentation that covers how to install and run the program, and organized our codebase to be more aligned with common best coding practices.
- We are in the process of building a datasheet to hold test results. This data will be collected for each of the four different file inputs. Gathering this data will allow us to progress towards implementing our optimizations.

Progress vs Plan

Planned Item	Status	Links	Notes/Blockers	Plan Change
Implement function testing	Partial	N/A	Still researching how to optimize the functions	Extend to next sprint
Documentation draft	Complete	GitHub	Outlines completed	None
Program flowchart analysis	Partial	N/A	Need to analyze the program's sequence of execution to determine how many times each function is called	Current priority
Prototype optimized functions	Not done	N/A	Explore and experiment possible program optimizations	None
Implement initial optimizations	Not done	N/A	Implement first program update	None
Add more sample data input	Not done	N/A	Waiting on project partner to return from out of office	None

Evidence of Working Software

CI/Build: [GitHub URL]()




```
==================================================
FUNCTION TIMING SUMMARY
==================================================
__init__: 6221065 calls, 1851.7135s total, 0.000298s avg
_preprocess_nav: 1 calls, 23.0702s total, 23.070232s avg
_preprocess_obs: 1 calls, 0.0002s total, 0.000246s avg
adjust_to_gps_time: 512098 calls, 0.4476s total, 0.000001s avg
calculate_positions: 1 calls, 565.8961s total, 565.896060s avg
copy: 622104 calls, 1.3400s total, 0.000002s avg
extract_datetime_information: 1 calls, 0.0005s total, 0.000529s avg
find_nearest_block: 285120 calls, 384.0126s total, 0.001347s avg
get_additional_fields: 285120 calls, 81.6512s total, 0.000286s avg
get_first_c: 285120 calls, 18.5274s total, 0.000065s avg
get_first_p: 285120 calls, 4.2180s total, 0.000015s avg
get_freq_band_rinex3: 757440 calls, 74.2369s total, 0.000098s avg
get_leap_second: 1152 calls, 0.0030s total, 0.000003s avg
get_satellite_position: 285120 calls, 501.8454s total, 0.001760s avg
get_satellite_position_glonass: 69120 calls, 127.0320s total, 0.001838s avg
get_satellite_position_std: 216000 calls, 347.4289s total, 0.001608s avg
glonass_diffeq: 2211936 calls, 11.8332s total, 0.000005s avg
has_freq_band: 1448640 calls, 4.9472s total, 0.000003s avg
load_config: 1 calls, 1847.4138s total, 1847.413762s avg
parse: 1 calls, 665.4933s total, 665.493254s avg
to_mjd: 296098 calls, 1.1188s total, 0.000004s avg
to_mjd_with_week_second: 216000 calls, 2.0703s total, 0.000010s avg
==================================================
```

Risk and Quality

Risk	Owner	Status	Concrete Actions
Code complexity	Team	Active	Modulating code sections, adding inline comments
Setup inconsistencies	Team	Resolved	Documented environment dependencies
Limited testing time	Team	Active	Prioritize optimizing the slowest functions
Poor documentation of function tests	Team	Active	Log pre-optimization function summaries and their expected outputs
Hidden dependencies between functions	Team	Active	Use trace debugging tools to observe the inner workings of the original program
Unclear success criteria	Team	Resolved	Log all implemented optimizations with time test results

Bug count: zero current known errors, extremely poor runtime efficiency.

Next Goals

- REQ-003: Enable runtime test outputs to be calculated with precision 1^{-10}.
- REQ-004: Test program on local environment and engineering server.
- REQ-008: Analyze program and create flowchart. Determine sequence of execution and the number of calls to each function.
- REQ-012: Add function headers and descriptions throughout the program to improve readability and reusability.

Team Process Reflection

Our last sprint consisted of getting comfortable with running the program on our local systems. Additionally, we used a runtime tool to record the processing speed of each function. For this sprint, we have focused on designing detailed tests we will need for implementing our optimizations. This includes checking if a CSV output is identical to the original output. We will test the 4 different input file combinations against the CSV match test and the runtime test. Logging our test results will put us in a good position to begin testing our optimized solutions.

Individual Contributions

Kathryn Butler

- Improved code structure on the repository.
- Drafted the Walking Skeleton Prototype.
  - Edited the video transcript.
  - Created run instructions.
  - Completed CI Health, Access, and Attn sections.

Michael Mcallister

- Added function speed benchmarking test to code base
- Identified bottlenecks in rnx2db.py
- Merged branch with main to include runtime benchmarking capabilities in main for future comparison with other branches where code changes will be implemented
- Made video for Walking Skeleton Prototype

Joseph Schaab

- Documented Progress Report Sprint 3
- Added Testing folder to repository
- Built datasheet to hold test results
- Outlined flowchart of rnx2db.py